

Programming concepts

Chris Paciorek

2025-10-01

Table of contents

Overview	2
1. Text manipulation, string processing and regular expressions (regex)	2
String processing and regular expressions in Python	2
Finding patterns	3
Manipulating and replacing patterns	5
Greedy matching	6
Special characters in Python	7
2. Reading data into Python	10
Core Python functions for text files	10
Connections and streaming	14
File paths	15
Reading data quickly: Arrow and Polars	15
3. Output from Python	16
Writing output to files	16
Formatting output	16
5. Interacting with the operating system and external code and configuring Python	17
Interacting with the operating system	17
Interacting with external code	21
6. Modules and packages	21
Modules	21
The import statement	22
Packages	23
Internal/private objects	24
Subpackages	25
Installing packages	26
Making your package installable (optional)	26
Reproducibility and package management	27
Package locations	28

Source vs. binary packages	28
7. Types and data structures	28
Standard data structures in Python and R	29
Other kinds of data structures	29
Types and classes	32
Overview and static vs. dynamic typing	32

Overview

This unit covers a variety of programming concepts, illustrated in the context of Python and with comments about and connections to other languages. It also serves as a way to teach some advanced features of Python. In general the concepts are relevant in other languages, though other languages may implement things differently. One of my goals for the unit is for us to think about why things are the way they are in Python. I.e., what principles were used in creating the language and what choices were made? While other languages use different principles and made different choices, understanding what one language does in detail will be helpful when you are learning another language or choosing a language for a project.

i Quarto configuration details for this document

This document uses the `knitr` engine for rendering to be able to run chunks in multiple languages (Python and bash, as well as a few R chunks. It has the Quarto configuration `ipynb-shell-interactivity: all`, so that output from all Python code in a chunk will print in the rendered document; when done with the `knitr` engine, the output is interspersed with the code in the chunk rather than printed all at the end.

1. Text manipulation, string processing and regular expressions (regex)

Text manipulations in Python have a number of things in common with UNIX, R, and Perl, as many of the ideas/software evolved from UNIX. When I use the term *string* here, I'll be referring to any sequence of characters that may include numbers, white space (including newlines), and special characters, usually stored as an object of the `str` class.

String processing and regular expressions in Python

Here we'll see functionality for working with strings in Python, focusing on regular expressions with the `re` package. This will augment our consideration of regular expressions in the shell, in particular by seeing how we can replace patterns in addition to finding them.

The `re` package provides Perl-style regular expressions, but it doesn't seem to support named character classes such as `[:digit:]`. Instead use classes such as `\d` and `[0-9]`.

Finding patterns

In Python, you can apply a matching function and then query the result to get information about what was matched and where in the string.

```
text = "Here's my number: 919-543-3300."  
m = re.search("\\d+", text)  
m
```

```
<re.Match object; span=(18, 21), match='919'>
```

```
m.group()
```

```
'919'
```

```
m.start()
```

```
18
```

```
m.end()
```

```
21
```

```
m.span()
```

```
(18, 21)
```

Notice that that showed us only the first match.

The [discussion of special characters](#) explains why we need to provide `\\d` rather than `\d`.

We can instead use `findall` to get all the matches.

```
re.findall("\\d+", text)
```

```
['919', '543', '3300']
```

This is equivalent to:

```
pattern = re.compile("\\d+")  
re.findall(pattern, text)
```

```
['919', '543', '3300']
```

The compile can be omitted and will be done implicitly, but is a good idea to do explicitly if you have a complex regex pattern that you will use repeatedly (e.g., on every line in a file). It is also a reminder that regular expressions is a separate language, which can be compiled into a program. The compilation results in an object that relies on finite state machines to match the pattern.

To ignore case, do the following:

```
text = "That cat in the Hat"  
re.findall("hat", text, re.IGNORECASE)
```

```
['hat', 'Hat']
```

There are several other regex flags (also called compilation flags) that can control the behavior of the matching engine in interesting ways (check out `re.VERBOSE` and `re.MULTILINE` for instance).

We can of course use list comprehension to work with multiple strings. But we need to be careful to check whether a match was found.

```
def return_group(pattern, txt):
    m = re.search(pattern, txt)
    if m:
        return m.group()
    else:
        return None

text = ["Here's my number: 919-543-3300.", "hi John, good to meet you",
        "They bought 731 bananas", "Please call 1.919.554.3800"]
[return_group("\\d+", str) for str in text]
```

```
['919', None, '731', '1']
```

Recall that we can search for location-specific matches in relation to the start and end of a string.

```
text = "hats are all that are important to a hatter."
re.findall("^hat\\w+", text)
```

```
['hats']
```

Recall that we can search based on repetitions (as already demonstrated with the `\\w+` just above).

```
text = "Here's my number: 919-543-3300. They bought 731 hats. Please call 1.919.554.3800."
re.findall("\\d{3}[-.]\\d{3}[-.]\\d{4}", text)
```

```
['919-543-3300', '919.554.3800']
```

As another example, the phone number detection problem could have been done a bit more compactly (as well as more generally to allow for an initial “1-” or “1.”) as:

```
text = "Here's my number: 919-543-3300. They bought 731 bananas. Please call 1.919.554.3800."
re.findall("((1[-.]?)\\d{3}[-.]){1,2}\\d{4}", text)
```

```
[('919-543-3300', '', '543-'), ('1.919.554.3800', '1.', '554.')]
```

Question: the above regex would actually match something that is not a valid phone number. What can go wrong?

When you are searching for all occurrences of a pattern in a large text object, it may be beneficial to use `finditer`:

```
it = re.finditer("(http|ftp):\\/\\/", text) # http or ftp followed by ://

for match in it:
    match.span()
```

This method behaves lazily and returns an iterator that gives us one match at a time, and

only scans for the next match when we ask for it. This is similar to the behavior we saw with `pandas.read_csv(chunksize = n)`

Manipulating and replacing patterns

We can replace matching substrings with `re.sub`.

```
text = "Here's my number: 919-543-3300."
re.sub("\\d", "Z", text)
```

"Here's my number: ZZZ-ZZZ-ZZZZ."

Next let's consider grouping using `()`. We'll see that the grouping operator also controls what is returned as the matched patterns.

Here's a basic example of using grouping via parentheses with the OR operator.

```
text = "At the site http://www.ibm.com. Some other text. ftp://ibm.com"
re.search("(http|ftp):\\/\\/\\/", text).group()
```

'http://'

However, if we want to find all the matches and try to use `findall`, we see that, when grouping operators are present, it returns only the “captured” groups, as discussed a bit in `help(re.findall)`, so we'd need to add an additional grouping operator to capture the full pattern when using `findall`:

```
re.findall("(http|ftp):\\/\\/\\/", text)
```

['http', 'ftp']

```
re.findall("((http|ftp):\\/\\/\\/)", text)
```

[('http://', 'http'), ('ftp://', 'ftp')]

If we only wanted to full pattern without capturing the inner group, we can use some additional syntax, `?:`, for “non-capturing” groups:

```
re.findall("(?:http|ftp):\\/\\/\\/)", text)
```

['http://', 'ftp://']

Groups are also used when we need to reference back to a detected pattern when doing a replacement. This is why they are sometimes referred to as “capturing groups”. For example, here we'll find any numbers and add underscores before and after them:

```
text = "Here's my number: 919-543-3300. They bought 731 bananas. Please call 919.554.3800."
re.sub("([0-9]+)", "_\\1_", text)
```

"Here's my number: _919_-_543_-_3300_. They bought _731_ bananas. Please call _919_.554_.3800_."

Here we'll remove commas not used as field separators.

```
text = '"H4NY07011","ACKERMAN, GARY L.,"H","$13,242",,,'
re.sub("[^\\",,)", "\\1", text)
```

```
'"H4NY07011","ACKERMAN GARY L.,"H","$13242",,,'
```

How does that work? Consider that `[^\",,]` matches a character that is not a quote and not a comma. The regex is such a character followed by a comma, with the matched character saved in `\1` because of the grouping operator.

💡 Challenge

Instead of removing the commas to remove the ambiguity, how would you convert the comma delimiters to pipe (`|`) delimiters (since the pipe is rarely used in text)?

Extending the use of `\1`, we can refer to multiple captured groups:

```
text = "Here's my number: 919-543-3300. They bought 731 bananas. Please call 919.554.3800."
re.sub("([0-9]{3})[-\.]([0-9]{3})[-\.]([0-9]{4})", "area code \1, number \2-\3", text)
```

```
"Here's my number: area code 919, number 543-3300. They bought 731 bananas. Please call area code 919
3800."
```

i Additional regex functionality

Regex extensions that we won't discuss further here include:

- Groups can also be given names, instead of having to refer to them by their numbers.
- We can have `sub` call a “callback” function to do the replacement. The function will be invoked on each match with the argument being equivalent to the result of `re.search`.
- We can reference previously captured groups within a pattern using the same `\1`-style syntax (as opposed to when doing replacement as seen above).

💡 Challenge

Suppose a text string has dates in the form “Aug-3”, “May-9”, etc. and I want them in the form “3 Aug”, “9 May”, etc. How would I do this regex?

Greedy matching

Finally, let's consider where a match ends when there is ambiguity.

As a simple example consider that if we try this search, we match as many digits as possible, rather than returning the first “9” as satisfying the request for “one or more” digits.

```
text = "See the 998 balloons."
re.findall("\\d+", text)
```

```
['998']
```

That behavior is called *greedy* matching, and it's the default. That example also shows why it is the default. What would happen if it were not the default?

However, sometimes greedy matching doesn't get us what we want.

Consider this attempt to remove multiple html tags from a string.

```
text = "Do an internship <b> in place </b> of <b> one </b> course."
re.sub("<.*>", "", text)
```

```
'Do an internship  course.'
```

Notice what happens because of greedy matching.

One way to avoid greedy matching is to use a `?` after the repetition specifier.

```
re.sub("<.*?>", "", text)
```

```
'Do an internship  in place  of  one  course.'
```

However, that syntax is a bit frustrating because `?` is also used to indicate 0 or 1 repetitions, making the regex a bit hard to read/understand.

Challenge

Suppose I want to strip out HTML tags but without using the `?` to avoid greedy matching. How can I be more careful in constructing my regex?

Special characters in Python

Recall that when characters are used for special purposes, we need to ‘escape’ them if we want them interpreted as the actual (*literal*) character. In what follows, I show this in Python, but similar manipulations are sometimes needed in the shell and in R.

This can get particularly confusing in Python as the backslash is also used to input special characters such as newline (`\n`) or tab (`\t`). Apparently there was some change in handling *escape sequences* as of Python 3.12. We now need to do this for the regex `\d`:

```
re.search("\\d+", "a93b")
```

```
<re.Match object; span=(1, 3), match='93'>
```

In Python 3.11, it was fine to use `\d`, but now we need `\\d`, because Python now tries to interpret `\d` as a special character (like `\n`, but `\d` doesn’t exist) and doesn’t pass it directly along as regex syntax.

Here are some examples of using special characters.

```
tmp = "Harry said, \"Hi\""
print(tmp)    # This prints out without a newline -- this is hard to show in rendered doc.
```

```
Harry said, "Hi"
```

```
tmp = "Harry said, \"Hi\".\n"
print(tmp)    # This prints out with the newline -- hard to show in rendered doc.
```

```
Harry said, "Hi".
```

```
tmp = ["azar", "foo", "hello\tthere\n"]
print(tmp[2])
```

```
hello    there
```

```
re.search("[\tZ]", tmp[2])    # Search for a tab or a 'Z'.
```

```
<re.Match object; span=(5, 6), match='\t'>
```

Here are some examples of using various special characters in regex syntax. To use a special character as a regular character, we need to escape it (which in Python 3.12 involves two backslashes, as discussed above):

```
## Search for characters that are not 'z'
## (using ^ as regular expression syntax)
re.search("[^z]", "zero")
```

```
<re.Match object; span=(1, 2), match='e'>
```

```
## Show results for various input strings:
for st in ["a^2", "93", "zzz", "zit", "azar"]:
    print(st + ":\t", re.search("[^z]", st))
```

```
a^2:    <re.Match object; span=(0, 1), match='a'>
93:    <re.Match object; span=(0, 1), match='9'>
zzz:    None
zit:    <re.Match object; span=(1, 2), match='i'>
azar:    <re.Match object; span=(0, 1), match='a'>
```

```
## Search for either a '^' (as a regular character) or a 'z':
for st in ["a^2", "93", "zzz", "zit", "azar"]:
    print(st + ":\t", re.search("[\\^z]", st))
```

```
a^2:    <re.Match object; span=(1, 2), match='^'>
93:    None
zzz:    <re.Match object; span=(0, 1), match='z'>
zit:    <re.Match object; span=(0, 1), match='z'>
azar:    <re.Match object; span=(1, 2), match='z'>
```

```
## Search for exactly three characters
## (using . as regular expression syntax)
for st in ["abc", "1234", "def"]:
    print(st + ":\t", re.search("^.{3}$", st))
```

```
abc:    <re.Match object; span=(0, 3), match='abc'>
1234:    None
def:    <re.Match object; span=(0, 3), match='def'>
```



```
## Search for a period (as a regular character)
for st in ["3.9", "27", "4.2"]:
    print(st + ":\t", re.search("\.", st))

3.9:      <re.Match object; span=(1, 2), match='.'>
27:      None
4.2:      <re.Match object; span=(1, 2), match='.'>
```

Challenge

Explain why we use a single backslash to get a newline and double backslash to write out a Windows path in the examples here:

```
## Suppose we want to use a \ in our string:
print("hello\nagain")
```

```
hello
again
```

```
print("hello\\nagain")
```

```
hello\nagain
```

```
print("My Windows path is: C:\\Users\\nadal.")
```

```
My Windows path is: C:\Users\nadal.
```

Another way to achieve this effect if your string does not contain any special characters is to prefix your string literal with an r for “raw”:

```
print(r"My Windows path is: C:\Users\nadal.")
```

```
My Windows path is: C:\Users\nadal.
```

On a more involved note, searching for an actual backslash gets even more complicated (you can search online for “[backslash plague](#)” or “backslash hell”), because we need to pass two backslashes as the regular expression, so that a literal backslash is searched for. However, to pass two backslashes, we need to escape each of them with a backslash so Python doesn’t treat each backslash as part of a special character. So that’s four backslashes to search for a single backslash! Yikes. One rule of thumb is just to keep entering backslashes until things work!

```
## Use and search for an actual backslash
tmp = "something \\ other\n"
print(tmp)
```

```
something \ other
```

```
re.search("\\\\", tmp)
```

```
<re.Match object; span=(10, 11), match='\\'>
```

```
try:
    re.search("\\", tmp)
except Exception as error:
    print(error)
```

bad escape (end of pattern) at position 0

Again here you can use “raw” strings, at the price of losing the ability to use any special characters:

```
## Search for an actual backslash
re.search(r"\\", tmp)
```

```
<re.Match object; span=(10, 11), match='\\'>
```

The use of the raw string `r"\"` tells Python to treat this string literal without any escaping, but that does not apply to the regex engine (or else we would have used a single backslash), so we do need the second backslash. So yes. This can be quite confusing.



Pasting quotation marks

Be careful when cutting and pasting from documents that are not text files as you may paste in something that looks like a single or double quote, but which Python cannot interpret as a quote because it's some other ASCII (or Unicode) quote character. If you paste in a ” from PDF, it will not be interpreted as a standard Python double quote mark.

Similar things come up in the shell and in R, but in the shell you often don't need as many backslashes. E.g. you could do this to look for a literal backslash character.

```
echo "hello" > file.txt
echo "with a \ there" >> file.txt
grep '\\' file.txt
```

```
with a \ there
```

2. Reading data into Python

The main downside to working with datasets in Python (true for R and most other languages as well) is that the entire dataset resides in memory, so using standard Python tools can be problematic in some cases; we'll discuss some alternatives later.

Core Python functions for text files

The `read_table` and `read_csv` functions in the Pandas package are commonly used for reading in data. They read in delimited files (CSV specifically in the latter case). The key arguments are the delimiter (the `sep` argument) and whether the file contains a header, a line with the variable names. We can use `read_fwf()` to read from a fixed width text file into a data frame.

The most difficult part of reading in such files can be dealing with how Pandas determines the types of

the fields that are read in. While Pandas will try to determine the types automatically, it can be safer (and faster) to tell Pandas what the types are, using the `dtype` argument to `read_table()`.

Let's work through a couple examples. Before we do that, let's look at the arguments to `read_table`. Note that `sep=''` can use regular expressions (which would be helpful if you want to separate on any amount of white space, as one example).

```
import os
import pandas as pd

dat = pd.read_table(os.path.join('.', 'data', 'RTADataSub.csv'),
                    sep = ',', header = None)
dat.dtypes.head()    # 'object' is string or mixed type

0    object
1    object
2    object
3    object
4    object
dtype: object

dat.loc[0,1]

'2336'

type(dat.loc[0,1]) # string!

<class 'str'>

## Whoops, there is an 'x', presumably indicating missingness:
dat.loc[:,1].unique()

array(['2336', '2124', '1830', '1833', '1600', '1578', '1187', '1005',
       '918', '865', '871', '860', '883', '897', '898', '893', '913',
       '870', '962', '880', '875', '884', '894', '836', '848', '885',
       '851', '900', '861', '866', '867', '829', '853', '920', '877',
       '908', '855', '845', '859', '856', '825', '828', '854', '847',
       '840', '873', '822', '818', '838', '815', '813', '816', '849',
       '802', '805', '792', '823', '808', '798', '800', '842', '809',
       '807', '826', '810', '801', '794', '771', '796', '790', '787',
       '775', '751', '783', '811', '768', '779', '795', '770', '821',
       '830', '767', '772', '791', '781', '773', '777', '814', '778',
       '782', '837', '759', '846', '797', '835', '832', '793', '803',
       '834', '785', '831', '820', '812', '824', '728', '760', '762',
       '753', '758', '764', '741', '709', '735', '749', '752', '761',
       '750', '776', '766', '789', '763', '864', '858', '869', '886',
       '844', '863', '916', '890', '872', '907', '926', '935', '933',
       '906', '905', '912', '972', '996', '1009', '961', '952', '981',
       '917', '1011', '1071', '1920', '3245', '3805', '3926', '3284',
       '2700', '2347', '2078', '2935', '3040', '1860', '1437', '1512',
```

```

        '1720', '1493', '1026', '928', '874', '833', '850', nan, 'x'],
        dtype=object)
## Let's treat 'x' as a missing value indicator.
dat2 = pd.read_table(os.path.join('.', 'data', 'RTADataSub.csv'),
                     sep = ',', header = None, na_values = 'x')
dat2.dtypes.head()

```

```

0    object
1    float64
2    float64
3    float64
4    float64
dtype: object

```

```

dat2.loc[:,1].unique()

```

```

array([2336., 2124., 1830., 1833., 1600., 1578., 1187., 1005.,  918.,
       865.,  871.,  860.,  883.,  897.,  898.,  893.,  913.,  870.,
       962.,  880.,  875.,  884.,  894.,  836.,  848.,  885.,  851.,
       900.,  861.,  866.,  867.,  829.,  853.,  920.,  877.,  908.,
       855.,  845.,  859.,  856.,  825.,  828.,  854.,  847.,  840.,
       873.,  822.,  818.,  838.,  815.,  813.,  816.,  849.,  802.,
       805.,  792.,  823.,  808.,  798.,  800.,  842.,  809.,  807.,
       826.,  810.,  801.,  794.,  771.,  796.,  790.,  787.,  775.,
       751.,  783.,  811.,  768.,  779.,  795.,  770.,  821.,  830.,
       767.,  772.,  791.,  781.,  773.,  777.,  814.,  778.,  782.,
       837.,  759.,  846.,  797.,  835.,  832.,  793.,  803.,  834.,
       785.,  831.,  820.,  812.,  824.,  728.,  760.,  762.,  753.,
       758.,  764.,  741.,  709.,  735.,  749.,  752.,  761.,  750.,
       776.,  766.,  789.,  763.,  864.,  858.,  869.,  886.,  844.,
       863.,  916.,  890.,  872.,  907.,  926.,  935.,  933.,  906.,
       905.,  912.,  972.,  996., 1009.,  961.,  952.,  981.,  917.,
      1011., 1071., 1920., 3245., 3805., 3926., 3284., 2700., 2347.,
      2078., 2935., 3040., 1860., 1437., 1512., 1720., 1493., 1026.,
       928.,  874.,  833.,  850.,   nan])

```

Using `dtype` is a good way to control how data are read in. Here's an example with a CSV file containing some HIV genome sequence information.

```

dat = pd.read_table(os.path.join('.', 'data', 'hivSequ.csv'),
                    sep = ',', header = 0,
                    dtype = {
                        'PatientID': int,
                        'Resp': int,
                        'PR Seq': str,
                        'RT Seq': str,
                        'VL-t0': float,

```

```
'CD4-t0': int})
dat.dtypes
```

```
PatientID      int64
Resp           int64
PR Seq         object
RT Seq         object
VL-t0          float64
CD4-t0         int64
dtype: object
```

```
dat.loc[0, 'PR Seq']
```

```
'CCTCAAATCACTCTTTGGCAACGACCCCTCGTCCCAATAAGGATAGGGGGGCAACTAAAGGAAGCYCTATTAGATACAGGAGCAGATGATACAGTATTAG
```

Note that you can avoid reading in one or more columns by using the `usecols` argument. Also, specifying the `dtype` argument explicitly should make for faster file reading.

If possible, it's a good idea to look through the input file in the shell or in an editor before reading into Python to catch such issues in advance. Using the UNIX command `less` on `RTADDataSub.csv` would have revealed these various issues, but note that `RTADDataSub.csv` is a 1000-line subset of a much larger file of data available from the [kaggle.com](https://www.kaggle.com) website. So more sophisticated use of UNIX utilities (as we will see in Unit 3) is often useful before trying to read something into a program.

If the file is not nicely arranged by field (e.g., if it has ragged lines), we'll need to do some more work. We can read each line as a separate string, after which we can process the lines using text manipulation. Here's an example from some US meteorological data where I know from metadata (not provided here) that the 4-11th values are an identifier, the 17-20th are the year, the 22-23rd the month, etc.

```
file_path = os.path.join('.', 'data', 'precip.txt')
with open(file_path, 'r') as file:
    lines = file.readlines()

id = [line[3:11] for line in lines]
year = [int(line[17:21]) for line in lines]
month = [int(line[21:23]) for line in lines]
nvalues = [int(line[27:30]) for line in lines]
year[0:5]
```

```
[2010, 2010, 2010, 2010, 2010]
```

Actually, that file, `precip.txt`, is in a “fixed-width” format (i.e., every element in a given column has the exact same number of characters), so reading in using `pandas.read_fwf()` would be a good strategy.

💡 Using with

The use of `with` above is the standard Python way to open files. It automatically closes the file after the body of the `with` statement finishes.

Connections and streaming

Python allows you to read in not just from a file but from a more general construct called a *connection*. This can include reading in text from the output of running a shell command and from unzipping a file on the fly.

Here are some examples of connections:

```
import gzip
with gzip.open('dat.csv.gz', 'r') as file:
    lines = file.readlines()

import zipfile
with zipfile.ZipFile('dat.zip', 'r') as archive:
    with archive.open('data.txt', 'r') as file:
        lines = file.readlines()

import subprocess
command = "ls -al"
output = subprocess.check_output(command, shell = True)
# `output` is a sequence of bytes.
with io.BytesIO(output) as stream: # Create a file-like object.
    content = stream.readlines()

df = pd.read_csv("https://download.bls.gov/pub/time.series/cu/cu.item", sep="\t")
```

If a file is large, we may want to read it in in chunks (of lines), do some computations to reduce the size of things, and iterate. This is referred to as online processing, streaming, or chunking, and can be done [using Pandas](#) (among other tools).

```
file_path = os.path.join('.', 'data', 'RTADataSub.csv')
chunksize = 50 # Obviously this would be much larger in any real application.

with pd.read_csv(file_path, chunksize = chunksize) as reader:
    for chunk in reader:
        # Manipulate the lines and store the key stuff.
        print(f'Read {len(chunk)} rows.')
```

More details on sequential (on-line) processing of large files can be found in the tutorial on large datasets mentioned in the reference list above.

One cool trick that can come in handy is to ‘read’ from a string as if it were a text file. Here’s an example:

```
import io

file_path = os.path.join('.', 'data', 'precip.txt')
with open(file_path, 'r') as file:
    text = file.read()

stringIOtext = io.StringIO(text)
df = pd.read_fwf(stringIOtext, header = None, widths = [3,8,4,2,4,2])
```

We can create connections for writing output too. Just make sure to open the connection first.

File paths

A few notes on file paths, related to ideas of reproducibility.

1. In general, you don't want to hard-code absolute paths into your code files because those absolute paths won't be available on the machines of anyone you share the code with. Instead, use paths relative to the directory the code file is in, or relative to a baseline directory for the project, e.g.:

```
dat = pd.read_csv('../data/cpds.csv')
```

2. Using UNIX style directory separators will work in Windows, Mac or Linux, but using Windows-style separators is not portable across operating systems.

```
## good: will work on Windows
dat = pd.read_csv('../data/cpds.csv')
## bad: won't work on Mac or Linux
dat = pd.read_csv('../data\cpds.csv')
```

3. Even better, use `os.path.join` so that paths are constructed specifically for the operating system the user is using:

```
## good: operating-system independent
dat = pd.read_csv(os.path.join('.', 'data', 'cpds.csv'))
```

Reading data quickly: Arrow and Polars

Apache Arrow provides efficient data structures for working with data in memory, usable in Python via the PyArrow package. Data are stored by column, with values in a column stored sequentially and in such a way that one can access a specific value without reading the other values in the column (O(1) lookup). Arrow is designed to read data from various file formats, including Parquet, native Arrow format, and text files. In general Arrow will only read data from disk as needed, avoiding keeping the entire dataset in memory.

Other options for avoiding reading all your data into memory include the Dask package and using `numpy.load` with the `mmap_mode` argument.

`polars` is designed to be a faster alternative to Pandas for working with data in-memory.

```
import polars
import time
t0 = time.time()
dat = pd.read_csv(os.path.join '..', 'data', 'airline.csv'))
t1 = time.time()
dat2 = polars.read_csv(os.path.join '..', 'data', 'airline.csv'), null_values = ['NA'])
t2 = time.time()
print(f"Timing for Pandas: {t1-t0}.")
```

Timing for Pandas: 2.3989484310150146.

```
print(f"Timing for Polars: {t2-t1}.")
```

Timing for Polars: 1.814377784729004.

3. Output from Python

Writing output to files

Functions for text output are generally analogous to those for input.

```
file_path = os.path.join('/tmp', 'tmp.txt')
with open(file_path, 'w') as file:
    file.writelines(lines)
```

We can also use `file.write()` to write individual strings.

In Pandas, we can use `DataFrame.to_csv` and `DataFrame.to_parquet`.

We can use the `json.dump` function to output appropriate data objects (e.g., dictionaries or possibly lists) as JSON. One use of JSON as output from Python would be to ‘serialize’ the information in an Python object such that it could be read into another program.

And of course you can always save to a Pickle data file (a binary file format) using `pickle.dump()` and `pickle.load()` from the `pickle` package. Happily this is platform-independent so can be used to transfer Python objects between different OS.

Formatting output

We can use [string formatting](#) to control how output is printed to the screen.

The mini-language involved in the format specification can get fairly involved, but a few basic pieces of syntax can do most of what one generally needs to do. This also feels like something an AI coding assistant could do well, though I haven’t specifically tried.

We can format numbers to chosen number of digits and decimal places and handle alignment, using the `format` method of the string class.

For example:


```

'{:>10}'.format(3.5)    # right-aligned, using 10 characters

'      3.5'

'{:.10f}'.format(1/3)    # force 10 decimal places

'0.3333333333'

'{:15.10f}'.format(1/3) # force 15 characters, with 10 decimal places

'    0.3333333333'

format(1/3, '15.10f') # alternative using a function

'    0.3333333333'

```

We can also “interpolate” variables into strings.

```

val1 = 1.5
val2 = 2.5
# This works as of Python 3.6.
print(f"Let's add {val1} and {val2}.")

```

Let's add 1.5 and 2.5.

```

num1 = 1/3
print("Let's add the %s numbers %.5f and %15.7f."
      %('floating point', num1, 32+1/7))

```

Let's add the floating point numbers 0.33333 and 32.1428571.

Or to insert into a file:

```

file_path = os.path.join('/tmp', 'tmp.txt')
with open(file_path, 'a') as file:
    file.write("Let's add the %s numbers %.5f and %15.7f."
              %('floating point', num1, 32+1/7))

```

round is another option, but it's often better to directly control the printing format.

5. Interacting with the operating system and external code and configuring Python

Interacting with the operating system

Scripting languages allow one to interact with the operating system in various ways. Most allow you to call out to the shell to run arbitrary shell code and save results within your session.

I'll assume everyone knows about the following functions/functionality for interacting with the filesystem and file in Python: `os.getcwd`, `os.chdir`, `import`, `pickle.dump`, `pickle.load`

Also in IPython there is additional functionality/syntax.

Here are a variety of tools for interacting with the operating system:

- To run UNIX commands from within Python, use `subprocess.run()`, as follows, noting that we can save the result of a system call to an Python object:

```
import subprocess, io
subprocess.run(["ls", "-al"])  ## results apparently not shown when compiled...
```

```
CompletedProcess(args=['ls', '-al'], returncode=0)
```

```
files = subprocess.run(["ls", "-al"], capture_output = True)
files.stdout
```

```
b'total 3180\ndrwxr-sr-x  9 paciorek scfstaff      55 Aug 19 07:31 .\ndrwxr-sr-
x 11 paciorek scfstaff      42 Aug 19 07:30 ..\n-rw-r--r--  1 paciorek scfstaff      803 Jun 30 1
-rw-r--r--  1 paciorek scfstaff 117142 Jun 30 14:19 chatgpt-regex-numbers.png\n-
-rw-r--r--  1 paciorek scfstaff  41130 Jun 30 14:19 cpds.csv\n-rw-r--r--  1 paciorek scfstaff 1
-rw-r--r--  1 paciorek scfstaff 1036183 Jun 30 14:18 file_nonascii.txt\n-rw-
r--r--  1 paciorek scfstaff  14610 Jun 30 14:19 gauss-seidel.png\n-rw-r--
r--  1 paciorek scfstaff      94 Jun 30 14:33 .github-access-token.txt\n-rw-r--
r--  1 paciorek scfstaff  20260 Jun 30 14:19 graph.png\n-rw-r--r--  1 paciorek scfstaff  2464
list.png\n-rw-r--r--  1 paciorek scfstaff      98 Jun 30 15:51 local.py\n-rw-
r--r--  1 paciorek scfstaff      79 Jun 30 15:44 mymod.py\ndrwxr-sr-x  4 paciorek scfstaff
-rw-r--r--  1 paciorek scfstaff  27757 Jun 30 14:19 nelder-mead.png\n-rw-r--
r--  1 paciorek scfstaff  14575 Jun 30 14:19 newest.json\n-rw-r--r--  1 paciorek scfstaff  226
-rw-r--r--  1 paciorek scfstaff  63998 Jun 30 14:19 normalized_example.png\ndrwxr-
sr-x  2 paciorek scfstaff      4 Jun 30 15:52 __pycache__\n-rw-r--r--  1 paciorek scfstaff  15
descent.png\n-rw-r--r--  1 paciorek scfstaff     111 Jun 30 15:51 test_scope.py\n-
-rw-r--r--  1 paciorek scfstaff      10 Jun 30 14:45 tmp2.txt\ndrwxr-sr-x  3 paciorek scfstaff
-rw-r--r--  1 paciorek scfstaff  51260 Jun 30 15:42 tmp.qmd\n-rw-r--r--  1 paciorek scfstaff  16
-rw-r--r--  1 paciorek scfstaff  51261 Aug 19 07:31 tmp.rmarkdown\n-rw-r--
r--  1 paciorek scfstaff      4 Jun 30 14:45 tmp.txt\n-rw-r--r--  1 paciorek scfstaff  9357 J
-rw-r--r--  1 paciorek scfstaff  88488 Jun 30 14:09 unit10-linalg.qmd\n-rw-
r--r--  1 paciorek scfstaff 111365 Jun 30 14:09 unit11-optim.qmd\ndrwxr-sr-
x  3 paciorek scfstaff      3 Aug 19 07:31 unit12-dl-optim_files\n-rw-r--
r--  1 paciorek scfstaff  83832 Aug 19 07:30 unit12-dl-optim.html\n-rw-r--
r--  1 paciorek scfstaff 112463 Aug 19 07:31 unit12-dl-optim.pdf\n-rw-r--
r--  1 paciorek scfstaff  26756 Jun 11 14:45 unit12-dl-optim.qmd\n-rw-r--
r--  1 paciorek scfstaff  25383 Jun 11 13:58 unit12-dl-optim.qmd~\ndrwxr-
sr-x  3 paciorek scfstaff      3 Aug 19 07:30 unit12-graphics_files\n-rw-
r--r--  1 paciorek scfstaff  75092 Aug 19 07:30 unit12-graphics.html\n-rw-
r--r--  1 paciorek scfstaff 1349340 Aug 19 07:30 unit12-graphics.pdf\n-rw-
r--r--  1 paciorek scfstaff  16568 Jun 30 14:09 unit12-graphics.qmd\n-rw-
r--r--  1 paciorek scfstaff  11498 Jun 30 14:09 unit1-intro.qmd\n-rw-r--
r--  1 paciorek scfstaff  27329 Jun 30 15:23 unit2-bash.qmd\n-rw-r--r--  1 paciorek scfstaff
bash.qmd~\n-rw-r--r--  1 paciorek scfstaff  49636 Jun 30 14:09 unit3-goodPractices.qmd\ndrwxr-
sr-x  4 paciorek scfstaff      4 Jun 30 15:55 unit4-programming_cache\ndrwxr-
```

```
sr-x 4 paciorek scfstaff      4 Jun 30 15:56 unit4-programming_files\n-rw-
r--r-- 1 paciorek scfstaff 167539 Jun 30 15:50 unit4-programming.qmd\n-rw-
r--r-- 1 paciorek scfstaff 152272 Jun 30 14:09 unit4-programming.qmd~\n-
rw-r--r-- 1 paciorek scfstaff 61063 Jun 30 14:09 unit5-bigData.qmd\n-rw-
r--r-- 1 paciorek scfstaff 49851 Jun 30 14:14 unit5-dataTech.qmd~\n-rw-
r--r-- 1 paciorek scfstaff 56877 Jun 30 14:09 unit6-parallel.qmd\n-rw-
r--r-- 1 paciorek scfstaff 36373 Jun 30 15:03 unit7-dataTech.qmd\n-rw-
r--r-- 1 paciorek scfstaff 31946 Jun 30 14:09 unit8-numbers.qmd\n-rw-r--
r-- 1 paciorek scfstaff 47904 Jun 30 14:09 unit9-sim.qmd\n-rw-r--r-- 1 paciorek scfstaff
rw-r--r-- 1 paciorek scfstaff 142 Jun 30 15:55 vec.py\n'
```

```
with io.BytesIO(files.stdout) as stream: # create a file-like object
    content = stream.readlines()
content[2:4]
```

```
[b'drwxr-sr-x 11 paciorek scfstaff      42 Aug 19 07:30 ..\n', b'-rw-r--r-- 1 paciorek scfstaff
```

- There are also a bunch of functions that will do specific queries of the filesystem, including

```
os.path.exists("unit2-dataTech.qmd")
```

False

```
os.listdir("../data")
```

```
['precipData.txt', 'co2_annmean_mlo.csv', 'hivSequ.csv', '0', 'airline.csv', 'stackoverflow-
2021.db', 'coop.txt.gz', 'precip.txt', 'test.r', 'RTADDataSub.csv', 'stackoverflow-
2021.duckdb', 'test.py', 'cpds.csv', 'airline.parquet']
```

- There are some tools for dealing with differences between operating systems. `os.path.join` is a nice example:

```
os.listdir(os.path.join("../", "data"))
```

```
['precipData.txt', 'co2_annmean_mlo.csv', 'hivSequ.csv', '0', 'airline.csv', 'stackoverflow-
2021.db', 'coop.txt.gz', 'precip.txt', 'test.r', 'RTADDataSub.csv', 'stackoverflow-
2021.duckdb', 'test.py', 'cpds.csv', 'airline.parquet']
```

It's best if you can to write your code, as shown here with `os.path.join`, in a way that is *agnostic* to the underlying operating system (i.e., that works regardless of the operating system).

- To get some info on the system you're running on:

```
import platform
platform.system()
```

'Linux'

```
os.uname()
```

```
posix.uname_result(sysname='Linux', nodename='smeagol', release='6.8.0-137-
generic', version='#137-Ubuntu SMP PREEMPT_DYNAMIC Fri Jul 17 20:28:23 UTC 2026', machine='x86_64')
```

```
platform.python_version()
```

```
'3.13.2'
```

```
sys.version
```

```
'3.13.2 | packaged by conda-forge | (main, Feb 17 2025, 14:40:20) [GCC 13.3.0]'
```

- To retrieve environment variables:

```
os.environ['PATH']
```

```
'/system/linux/miniforge-3.13/bin:/usr/local/linux/miniforge-3.13/condabin:/system/linux/minifor  
3.13/condabin:/system/linux/miniforge-3.13/bin:/system/linux/miniforge-3.13/condabin:/system/lin  
3.13/bin:/system/linux/miniforge-3.13/condabin:/system/linux/miniforge-3.13/bin:/accounts/vis/pa  
1.10.4/bin:/usr/local/linux/miniforge-3.13/bin:/accounts/vis/paciorek/bin:/usr/local/linux/bin:/  
server/bin:/accounts/vis/paciorek/.local/bin'
```

- You can have a Python script act as a shell script (like running a bash shell script) as follows.
 1. Write your Python code in a text file, say `example.py`
 2. As the first line of the file, include `#!/usr/bin/python` (like `#!/bin/bash` in a bash shell file, as seen in Unit 2) or for more portability across machines, include `#!/usr/bin/env python`.
 3. Make the Python code file executable with `chmod`: `chmod ugo+x example.py`.
 4. Run the script from the command line: `./example.py`

If you want to pass arguments into your script, you can do so with the `argparse` package.

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument('-y', '--year', default=2002,
                    help='year to download')
parser.add_argument('-m', '--month', default=None,
                    help='month to download')
args = parser.parse_args()
args.year
year = int(args.year)
```

Now we can run it as follows in the shell:

```
./example.py 2004 January
```

- Use `Ctrl-C` to interrupt execution. This will generally back out gracefully, returning you to a state as if the command had not been started. Note that if Python is exceeding the amount of memory available, there can be a long delay. This can be frustrating, particularly since a primary reason you would want to interrupt is when Python runs out of memory.

Interacting with external code

Scripting languages such as R, Python, and Julia allow you to call out to “external code”, which often means C or C++ (but also Fortran, Java and other languages).

Calling out to external code is particularly important in languages like R and Python that are often much slower than compiled code and less important in a fast language like Julia (which uses Just-In-Time compilation – more on that later).

In fact, the predecessor language to R, which was called ‘S’ was developed specifically (at AT&T’s Bell Labs in the 1970s and 1980s) as an interactive wrapper around Fortran, the numerical programming language most commonly used at the time (and still widely relied on today in various legacy codes).

In Python, one can [directly call out to C or C++ code](#) or one can use *Cython* to interact with C. With Cython, one can:

- Have Cython automatically translate Python code to C, if you provide type definitions for your variables.
- Define C functions that can be called from your Python code.

In R, one can call directly out to C or C++ code using *.Call* or one can use the [Rcpp package](#). *Rcpp* is specifically designed to be able to write C++ code that feels somewhat like writing R code and where it is very easy to pass data between R and C++.

6. Modules and packages

Scripting languages that become popular generally have an extensive collection of add-on packages available online (the causal relationship of the popularity and the extensive add-on packages goes in both directions).

A big part of Python’s popularity is indeed the extensive collection of add-on packages on [PyPI](#) (and GitHub and elsewhere) and via **Conda** that provide much of Python’s functionality (including core numerical capabilities via **numpy** and **scipy**).

To make use of a package it needs to be installed on your system (using **pip install** or **conda install**) *once* and loaded into Python (using the **import** statement) *every time you start a new session*.

Some modules are *installed* by default with Python (e.g., **os** and **re**), but all need to be loaded by the user in a given Python session.

Modules

A *module* is a collection of related code in a file with the extension **.py**. The code can include functions, classes, and variables, as well as runnable code. To access the objects in the module, you need to import the module.

Here we’ll create **mymod.py** from the shell, but of course usually one would create it in an editor.

```
cat << EOF > mymod.py
x = 7
range = 3
```

```
def myfun(x):
    print("The arg is: ", str(x), ".", sep = '')
EOF
```

```
import mymod
print(mymod.x)
```

7

```
mymod.myfun(99)
```

The arg is: 99.

The import statement

The import statement allows one to get access to code in a module. Importantly it associates the names of the objects in the module with a name accessible in the scope in which it was imported (i.e., the current context). The mapping of names (references) to objects is called a *namespace*. We discuss scopes and namespaces in more detail later.

```
del mymod
try:          # Check if `mymod` is in scope.
    mymod.x
except Exception as error:
    print(error)
```

name 'mymod' is not defined

```
y = 3
```

```
import mymod
mymod          # This is essentially a dictionary in the current (global) scope.
```

```
<module 'mymod' from '/accounts/vis/paciorek/teaching/243fall26/fall-2026/units/mymod.py'>
```

```
x          # This is not a name in the current (global) scope.
```

NameError: name 'x' is not defined

```
range          # This is a builtin, not from the module.
```

```
<class 'range'>
```

```
mymod.x
```

7

```
dir(mymod)
```

```
['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__']
```

```
mymod.x
```

7

```
mymod.range
```

3

So `y` and `mymod` are in the global namespace and `range` and `x` are in the module namespace of `mymod`. You can access the built-in `range` function from the global namespace but it turns out it's actually in the built-ins scope (more later).

Note the usefulness of distinguishing the objects in a module from those in the global namespace. We'll discuss this more in a bit.

That said, we can make an object defined in a module directly accessible in the current scope (adding it to the global namespace in this example) at which point it is distinct from the object in the module:

```
from mymod import x
x # now part of global namespace
```

7

```
dir()
```

```
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', 'mymod.x']
```

```
mymod.x = 5
```

```
x = 3
```

```
mymod.x
```

5

```
x
```

3

But in general we wouldn't want to use `from` to import objects in that fashion because we could introduce name *conflicts* and we reduce modularity.

That said, it can be tedious to always have to type the module name (and in many cases there are multiple submodule names you'd also need to type).

```
import mymod as m
m.x
```

5

Packages

A package is a directory containing one or more modules and with a file named `__init__.py` that is called when a package is imported and serves to initialize the package.

Let's create a basic package.

```
mkdir mypkg

cat << EOF > mypkg/__init__.py
## Make objects from mymod.py available as mypkg.foo rather than mypkg.mymod.foo.
## The "." is a "relative" import that means find "mymod" here in this directory.
from .mymod import *

print("Welcome to my package.")
EOF

cat << EOF > mypkg/mymod.py
x = 7

def myfun(val):
    print(f"Converting {val} to integer: {int(val)}.")
EOF
```

Note that if there were other modules, we could have imported from those as well.

Now we can use the objects from the module without having to know that it was in a particular module (because of how `__init__.py` was set up).

```
import mypkg
```

```
Welcome to my package.
```

```
mypkg.x
```

```
7
```

```
mypkg.myfun(7.3)
```

```
Converting 7.3 to integer: 7.
```

Note, one can set `__all__` in an `__init__.py` to define what is imported, which makes clear what is publicly available and hides what is considered internal.

Internal/private objects

We could add another module that the main module uses but that is not intended for direct use by the user. Here we name the function to start with `_` following the convention that this indicates a private/internal function.

```
cat << EOF > mypkg/auxil.py

def _helper(val):
    return val + 10
EOF
```



```
cat << EOF >> mypkg/mymod.py

from .auxil import _helper

def myfun10(val):
    print(f"Converting {val} to integer plus 10: {int(_helper(val))}.")
EOF
```

```
del mypkg
import mypkg
mypkg.myfun10(7.3)
```

Converting 7.3 to integer plus 10: 17.

```
del mypkg
```

Subpackages

Packages can also have modules in nested directories, achieving additional modularity via *subpackages*. A package can automatically import the subpackages via the main `__init__.py` or require the user to import them manually, e.g., `import mypkg.mysubpkg`.

```
mkdir mypkg/mysubpkg
```

```
cat << EOF > mypkg/mysubpkg/__init__.py
from .values import *
print("Welcome to my package's subpackage.")
EOF
```

```
cat << EOF > mypkg/mysubpkg/values.py
x = 999
b = 7
_my_internal_var = 9
EOF
```

```
import mypkg.mysubpkg      ## Note that __init__.py is invoked
```

Welcome to my package's subpackage.

```
mypkg.mysubpkg.b
```

```
7
```

```
mypkg.x
```

```
7
```

Note that a given `__init__.py` is invoked when importing anything nested within the directory containing the `__init__.py`; in the case above the `__init__.py` from `mypkg` is invoked, though for some reason the “Welcome to my package.” output is not showing up in this rendered document.

If we wanted to automatically import the subpackage we would add `from . import mysubpkg` to `mypkg/__init__.py`, which uses [relative imports](#). The alternative “absolute” import would be `import mypkg.mysubpkg`, which finds `mypkg` using `sys.path` (which specifies a set of paths of where to look).

One would generally not import the items from `mysubpkg` directly into the `mypkg` namespace but there may be cases one would do something like this. For example `numpy.linspace` is actually found in `numpy/core/function_base.py`, but we don’t need to refer to `numpy.core.linspace` because of how `numpy` structures the `import` statements in `__init__.py`. In contrast, the linear algebra functions are available via the subpackage namespace as `numpy.linalg.<function_name>`.

Take a look at `dir(numpy)`, `dir(numpy.linalg)`, and `dir(numpy.core)` to get a better sense for this in a real package.

Installing packages

If a package is on PyPI or available through Conda but not on your system, you can install it easily (usually). You don’t need root permission on a machine to install a package, though you may need to use `pip install --user` or set up a new Conda environment.

Packages often depend on other packages. In general, if one package depends on another, `pip` or `conda` will generally install the dependency automatically.

One advantage of Conda is that it can also install non-Python packages on which a Python package depends, whereas with `pip` you sometimes need to install a system package to satisfy a dependency.

i Using Mamba/libmamba and the Conda-forge channel

It’s not uncommon to run into a case where `conda` has trouble installing a package because of version inconsistencies amongst the dependencies. `mamba` is a drop-in replacement for `conda` and often does a better job of this “dependency resolution”. [We use mamba by default on the SCF](#). In recent versions of Conda, you can also use the Mamba’s dependency resolver when running `conda` commands by running `conda config --set solver libmamba`, which puts `solver: libmamba` in your `.condarc` file. It’s also generally recommended to use the `conda-forge channel` (i.e., location) when installing packages with Conda (this is done automatically when using `mamba`). `conda-forge` provides a wide variety of up-to-date packages, maintained by the community.

Making your package installable (optional)

It’s pretty easy to configure your package so that it can be built and installed via `pip`. See the structure of [this example repository](#). In fact, one can install the package with only either `setup.py` or `pyproj.toml`, but the other files listed here are recommended:

- `pyproj.toml` (or `pyproject.toml`): this is a configuration file used by packaging tools. In the `mytoy` example it specifies to use `setuptools` to build and install the package.
- `setup.py`: this is run when the package is built and installed when using `setuptools`. In the example, it simply runs `setuptools.setup()`. With recent versions of `setuptools`, you don’t actually need this so long as you have the `pyproj.toml` file.
- `setup.cfg`: provides metadata about the package when using `setuptools`.

- `environment.yml`: provides information about the full environment in which your package should be used (including examples, documentation, etc.). For projects using `setuptools`, a minimal list of dependencies needed for installation and use of the package can instead be included in the `install_requires` option of `setup.cfg`.
- `LICENSE`: specifies the license for your package giving the terms under which others can use it.

The `postBuild` file is a completely optional file only needed if you want to use the package with a MyBinder environment.

At the [numpy GitHub repository](#), by looking in `pyproject.toml`, you can see that `numpy` is build and installed using a system called *Meson*, while at the [Jupyter GitHub repository](#) you can see that the `jupyter` package is built and installed using `setuptools`.

Building a package usually refers to compiling source code but for a Python package that just has Python code, nothing needs to be compiled. *Installing* a package means putting the built package into a location on your computer where packages are installed.

You can also make your package public on PyPI or through Conda, but that is not something we'll cover here.

Reproducibility and package management

For reproducibility, it's important to know the versions of the packages you use (and the version of Python). `pip` and `conda` make it easy to do this. You can create a *requirements* file that captures the packages you are currently using (and, critically, their versions) and then install exactly that set of packages (and versions) based on that requirements file.

```
pip freeze > requirements.txt
pip install -r requirements.txt

conda env export --no-builds > environment.yml
conda env create -f environment.yml
```

The `--no-builds` flag make sure not to include information about the dependencies that are system-specific (e.g., that would only work on MacOS and not on Windows).

Conda is a general package manager. You can use it to manage Python packages but lots of other software as well, including R and Julia.

Conda environments provide an additional layer of modularity/reproducibility, allowing you to set up a fully reproducible environment for your computation. Here (by explicitly giving `python=3.13`) the Python 3.13 executable and all packages you install in the environment are fully independent of whatever Python executables are installed on the system.

```
conda create -n myenv python=3.13
source activate myenv
conda install numpy
```

Warning

If you use `conda activate` rather than `source activate`, Conda will prompt you to run `conda init`, which will make changes to your `~/.bashrc` that, for one, activate the Conda base environment automatically when a shell is started. This may be fine, but it's helpful to be aware.

Package locations

Packages in Python (and in R, Julia, etc.) may be installed in various places on the filesystem, and it sometimes it is helpful (e.g., if you end up with multiple versions of a package installed on your system) to be able to figure out where on the filesystem the package is being loaded from.

We can use the `__file__` and `__version__` objects in a package to see where on the filesystem a package is installed and what version it is:

```
import numpy as np
np.__file__

'/system/linux/miniforge-3.13/lib/python3.13/site-packages/numpy/__init__.py'

np.__version__

'2.1.3'
```

(`pip list` or `conda list` will also show version numbers, for all packages.)

`sys.path` shows where Python looks for packages on your system.

Source vs. binary packages

The difference between a *source* package and a *binary* package is that the source package has the raw Python (and C/C++ and Fortran, in some cases) code as text files, while the binary package has all the non-Python code in a binary/non-text format, with the C/C++ and Fortran code already having been compiled.

If you install a package from source, C/C++/Fortran code will be compiled on your system (if the package has such code). That should mean the compiled code will work on your system, but requires you to have a compiler available and things properly configured. A binary package doesn't need to be compiled on your system, but in some cases the code may not run on your system because it was compiled in such a way that is not compatible with your system.

Python *wheels* are a binary package format for Python packages. Wheels for some packages will vary by platform (i.e., operating system) so that the package will install correctly on the system where it is being installed.

7. Types and data structures

As we're reading data into Python or other languages, it's important to think about the data structures we'll use to store the information. The data structure we choose can affect:

- the amount of memory we need,
- how quickly we can access the information in the data structure,
- how much copying needs to be done to add information to or remove information from the data structure,
- how efficiently we can use the data in subsequent computations.

This means that what you plan to do with the data should guide what kind of structure you store the data in.

Standard data structures in Python and R

- In Python and R, one often ends up working with dataframes, lists, and arrays/vectors/matrices/tensors.
- In Python we commonly work with data structures that are part of additional packages, in particular numpy arrays and pandas dataframes.
- Dictionaries in Python allow for easy use of key-value pairs where one can access values based on their key/label. In R one can do something similar with named vectors or named lists or (more efficiently) by using environments.
- In R, if we are not working with rectangular datasets or standard numerical objects, we often end up using lists or enhanced versions of lists, sometimes with deeply nested structures.

In Unit 6, we'll talk about *distributed* data structures that allow one to easily work with data distributed across multiple computers.

Other kinds of data structures

You may have heard of various other kinds of data structures, such as linked lists, trees, graphs, queues, and stacks. One of the key aspects that differentiate such data structures is how one navigates through the elements.

Sets are collections of elements that don't have any duplicates (like a mathematical set).

With a *linked list*, with each element (or node) has a value and a pointer (reference) to the location of the next element. (With a doubly-linked list, there is also a pointer back to the previous element.) One big advantage of this is that one can insert an element by simply modifying the pointers involved at the site of the insertion, without copying any of the other elements in the list. A big disadvantage is that to get to an element you have to navigate through the list.

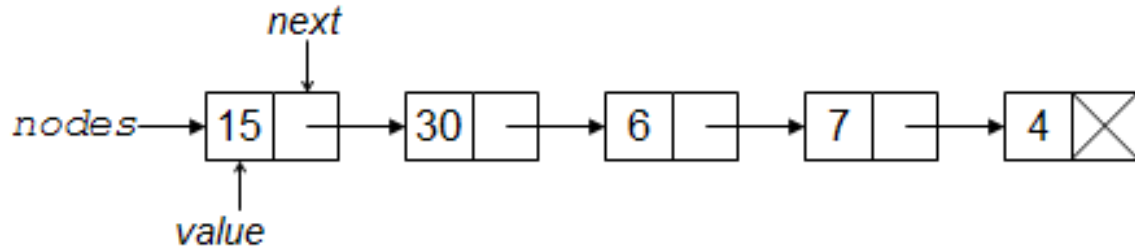


Figure 1: Linked list (courtesy of computersciencewiki.org)

Both *trees* and *graphs* are collections of nodes (vertices) and links (edges). A tree involves a set of nodes and links to child nodes (also possibly containing information linking the child nodes to their parent nodes). With a graph, the links might not be directional, and there can be cycles.

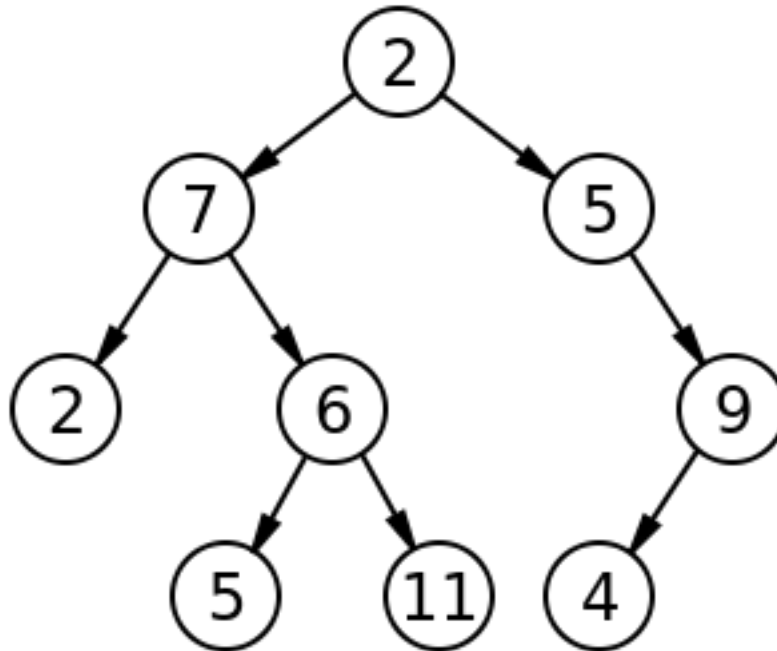


Figure 2: Tree (courtesy of computersciencewiki.org)

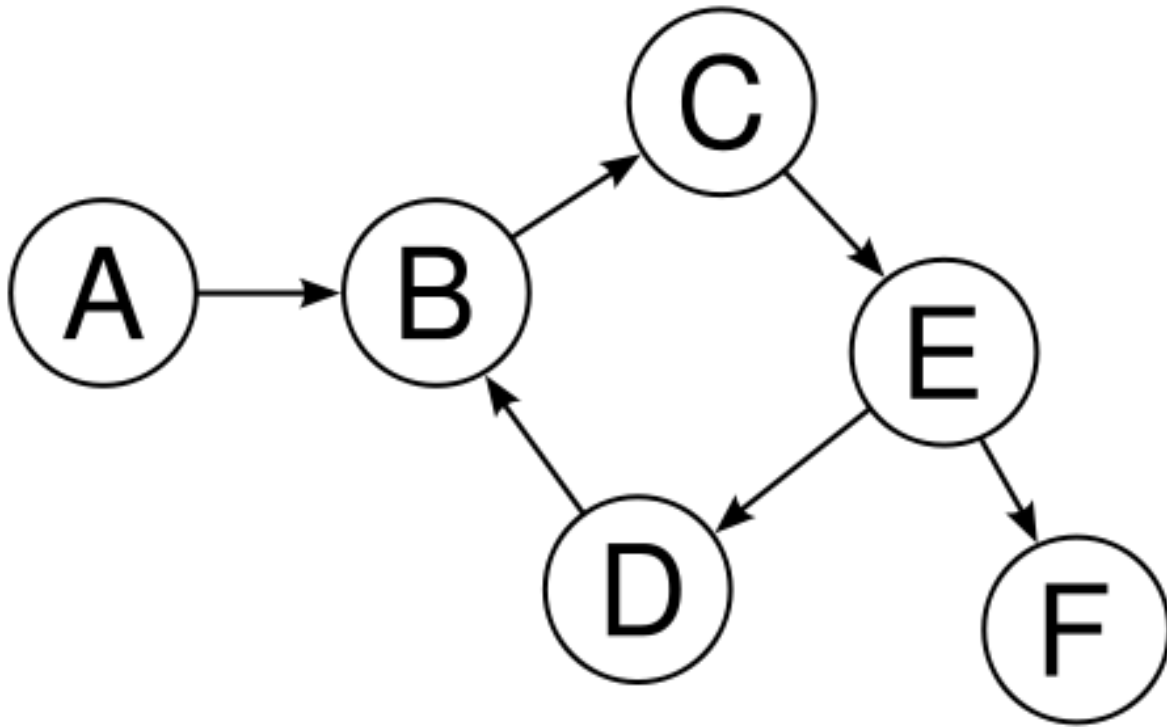


Figure 3: Graph (courtesy of computersciencewiki.org)

A *stack* is a collection of elements that behave like a stack of lunch trays. You can only access the top element directly (“last in, first out”), so the operations are that you can push a new element onto the stack or pop the top element off the stack. In fact, nested function calls behave as stacks, and the memory used in the process of evaluating the function calls is called the ‘stack’.

A *queue* is like the line at a grocery store, behaving as “first in, first out”.

One can use such data structures either directly or via add-on packages in Python and R, though I don’t think they’re all that commonly used in R. This is probably because statistical/data science/machine learning workflows often involve either ‘rectangular’ data (i.e., dataframe-style data) and/or mathematical computations with arrays. That said, trees and graphs are widely used.

Some related concepts that we’ll discuss further in Unit 5 include:

- **types:** this refers to how a given piece of information is stored and what operations can be done with the information.
 - ‘primitive’ types are the most basic types that often relate directly to how data are stored in memory or on disk (e.g., booleans, integers, numeric (real-valued), character, pointer (address, reference)).
- **pointers:** references to other locations (addresses) in memory. One often uses pointers to avoid unnecessary copying of data.

- hashes: hashing involves fast lookup of the value associated with a key (a label), using a hash function, which allows one to convert the key to an address. This avoids having to find the value associated with a specific key by looking through all the keys until the key of interest is found (an $O(n)$ operation).

Types and classes

Overview and static vs. dynamic typing

The term ‘type’ refers to how a given piece of information is stored and what operations can be done with the information.

‘Primitive’ types are the most basic types that often relate directly to how data are stored in memory or on disk (e.g., boolean, integer, numeric (real-valued, aka *double* or *floating point*), character, pointer (aka *address*, *reference*)).

In compiled languages like C and C++, one has to define the type of each variable. Such languages are *statically* typed. Interpreted (or scripting) languages such as Python and R have *dynamic* types. One can associate different types of information with a given variable name at different times and without declaring the type of the variable:

```
x = 'hello'
print(x)
```

```
hello
```

```
x*3
```

```
'hellohellohello'
```

```
x = 7
```

```
x*3
```

```
21
```

```
x = {'mykey': 7}
```

```
x*3
```

```
TypeError: unsupported operand type(s) for *: 'dict' and 'int'
```

In contrast in a language like C, one has to declare a variable based on its type before using it:

```
double y;
double x = 3.1;
y = x * 7.1;
```

Dynamic typing can be quite helpful from the perspective of quick implementation and avoiding tedious type definitions and problems from minor inconsistencies between types (e.g., multiplying an integer by a real-valued number). But static typing has some critical advantages from the perspective of software development, including:

- protecting against errors from mismatched values and unexpected user inputs, and

- generally much faster execution because the type of a variable does not need to be checked (and the location of the value found) when the code is run.

More complex types in Python (and in R) often use references (*pointers*, aka *addresses*) to the actual locations of the data. We'll see this in detail when we discuss Memory.